

TP 4 — Spark SQL : requêter le fil rouge e-commerce

Big Data Engineering — Master 1 — DMI/FST/UCAD — Prof. Samba Ndiaye

Consignes

- Complétez toutes les cellules marquées `# === À COMPLÉTER ===` (remplacez les `...`).
- Rédigez vos réponses dans les cellules *Votre réponse* :
- Le notebook doit s'exécuter **de bout en bout** (Kernel > Restart & Run All) avant d'être poussé.
- Livrable : `notebooks/TP4_spark_sql.ipynb` **avec les sorties visibles**, poussé sur votre dépôt avant la séance 5.

Déroulé

Partie	Contenu	Durée
A	Mise en place : données, SparkSession, vues	15 min
B	Premières requêtes SQL	25 min
C	L'enquête FCFA	30 min
D	Les indicateurs de la direction	40 min
E	SQL ou API ? <code>explain()</code> tranche	20 min
F	Discussion et quiz	20 min

0. Vérification de l'environnement

Données : si le dossier `data/` est absent, exécutez d'abord dans un terminal (ou une cellule `!`) :

```
python generate_data.py --scale 0.1 --outdir data
```

Graine 42 : tous les étudiants ont **exactement** les mêmes données.

```
In [1]: import os

os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"
os.environ["SPARK_LOCAL_HOSTNAME"] = "localhost"
os.environ.pop("SPARK_HOME", None)

import sys
os.environ["PYSPARK_PYTHON"] = sys.executable
```

```

os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

import pyspark
from pyspark.sql import SparkSession

local_spark_temp = r"C:\tmp\spark"
os.makedirs(local_spark_temp, exist_ok=True)
os.environ["TMP"] = local_spark_temp
os.environ["TEMP"] = local_spark_temp

print("Python :", sys.version)
print("PySpark :", pyspark.__version__)

if "spark" in globals():
    try:
        spark.stop()
    except Exception:
        pass

spark = (
    SparkSession.builder
    .appName("TP4-SparkSQL")
    .master("local[1]")
    .config("spark.driver.host", "127.0.0.1")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.local.dir", local_spark_temp)
    .config("spark.sql.ansi.enabled", "false")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")
print("Spark :", spark.version)

```

Python : 3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]

PySpark : 4.2.0

c:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\venv-bigdata\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.

require_minimum_pandas_version()

Spark : 4.2.0

Tableau de relevés

Il se remplit **au fil du TP** ; la dernière cellule du notebook l'affiche. Un notebook sans chiffres n'est pas un livrable.

```

In [2]: releves = {
    "A_nb_lignes_clients": None,
    "A_nb_lignes_commandes": None,
    "A_type_montant_total_fcfa": None, # ex. "string"
    "C2_nb_valeurs_polluees": None,
    "C1_ca_naif": None,
    "C4_ca_nettoyee": None,
    "C5_ecart_fcfa": None,
    "C5_ecart_pct": None,
    "D1_part_ca_livree_pct": None,
    "D2_mois_record": None,

```

```
"D3_panier_moyen_mobile":    None,
"D5_part_mobile_money_pct":  None,
}
```

Partie A — Mise en place (15 min)

A.1 — Charger les quatre sources et créer les vues

Chargez `customers.csv`, `orders.csv`, `products.csv` (CSV : `header=True`, `inferSchema=True`) et `payments.json`, puis créez les vues temporaires `clients`, `commandes`, `produits`, `paiements`.

```
In [3]: base = "../data/"

clients = spark.read.csv(base + "customers.csv", header=True, inferSchema=True)
commandes = spark.read.csv(base + "orders.csv", header=True, inferSchema=True)
produits = spark.read.csv(base + "products.csv", header=True, inferSchema=True)
paiements = spark.read.json(base + "payments.json")

clients.createOrReplaceTempView("clients")
commandes.createOrReplaceTempView("commandes")
produits.createOrReplaceTempView("produits")
paiements.createOrReplaceTempView("paiements")

spark.catalog.listTables()
```

```
Out[3]: [Table(name='clients', catalog=None, namespace=[], description=None, tableType='TEMPORARY', isTemporary=True),
Table(name='commandes', catalog=None, namespace=[], description=None, tableType='TEMPORARY', isTemporary=True),
Table(name='paiements', catalog=None, namespace=[], description=None, tableType='TEMPORARY', isTemporary=True),
Table(name='produits', catalog=None, namespace=[], description=None, tableType='TEMPORARY', isTemporary=True)]
```

A.2 — Premier relevé

Comptez les lignes de `clients` et `commandes`, affichez le schéma de `commandes`, et relevez le **type inféré** de `montant_total_fcfa` et de `frais_livraison_fcfa`.

```
In [17]: # === À COMPLÉTER ===
relevés["A_nb_lignes_clients"] = spark.sql("SELECT COUNT(*) AS n FROM clients")
relevés["A_nb_lignes_commandes"] = spark.sql("SELECT COUNT(*) AS n FROM commande")

commandes.printSchema()
relevés["A_type_montant_total_fcfa"] = dict(commandes.dtypes)["montant_total_fcfa"]
print(relevés)
```

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- date_commande: timestamp (nullable = true)
|-- statut: string (nullable = true)
|-- canal: string (nullable = true)
|-- frais_livraison_fcfa: integer (nullable = true)
|-- montant_total_fcfa: string (nullable = true)
```

```
{'A_nb_lignes_clients': 5025, 'A_nb_lignes_commandes': 50000, 'A_type_montant_total_fcfa': 'string', 'C2_nb_valeurs_polluees': None, 'C1_ca_naif': None, 'C4_ca_nettoyé': None, 'C5_ecart_fcfa': None, 'C5_ecart_pct': None, 'D1_part_ca_livree_pct': None, 'D2_mois_record': None, 'D3_panier_moyen_mobile': None, 'D5_part_mobile_money_pct': None}
```

Question A — Une des deux colonnes de montants n'a pas le type attendu. Laquelle, et qu'en déduisez-vous sur le contenu du fichier ? (Vous vérifierez votre hypothèse en partie C.)

Votre réponse :

montant_total_fcfa est en string, alors que frais_livraison_fcfa est correctement inféré en integer. C'est la colonne anormale.

On peut en déduire que le fichier orders.csv contient, dans la colonne montant_total_fcfa, au moins une valeur qui n'est pas un nombre pur. Ce qui force Spark à typer toute la colonne en string par sécurité (dès qu'une seule ligne ne peut pas être interprétée comme numérique, inferSchema bascule l'ensemble de la colonne). Les causes les plus courantes pour ce genre de TP "données sales" : des valeurs vides ou marquées "N/A" / "NULL" mélangées à des nombres, un séparateur décimal incohérent (, au lieu de ., ou l'inverse), des espaces ou symboles parasites collés au nombre (ex. "12 500", "12,500 FCFA") et des guillemets ou caractères invisibles autour de la valeur.

Partie B — Premières requêtes SQL (25 min)

Une requête par cellule, résultat affiché avec `.show()`.

B1 — Les 10 premiers clients de Dakar

Colonnes : `customer_id`, `prenom`, `nom`, `ville`.

```
In [18]: # === À COMPLÉTER ===
spark.sql("""
    SELECT customer_id, prenom, nom, ville
    FROM clients
    WHERE ville = 'Dakar'
    LIMIT 10
    """).show()
```

```

+-----+-----+-----+-----+
|customer_id| prenom|      nom|ville|
+-----+-----+-----+-----+
|    C000878|  Yacine|    Faye|Dakar|
|    C002485|   Astou|   Ndiaye|Dakar|
|    C000812|   Diarra|    Wade|Dakar|
|    C000249|Seynabou|Goudiaby|Dakar|
|    C003299|   Adama|    Fall|Dakar|
|    C001282|  Yacine|     Sy|Dakar|
|    C000334|Maguette|   Mendy|Dakar|
|    C002548| Rokhaya|   Badji|Dakar|
|    C002842|    Omar|   Diallo|Dakar|
|    C001159| Sokhna|    Dieng|Dakar|
+-----+-----+-----+-----+

```

B2 — Combien de villes distinctes dans clients ?

```

In [8]: # === À COMPLÉTER ===
spark.sql("""
    SELECT COUNT(DISTINCT ville) AS nb_villes
    FROM clients
    """).show()

```

```

+-----+
|nb_villes|
+-----+
|         56|
+-----+

```

B3 — Top 10 des produits les plus chers

Nom et prix, tri décroissant.

```

In [20]: spark.sql("""
    SELECT nom_produit,
           prix_unitaire_fcfa AS prix_fcfa
    FROM produits
    ORDER BY prix_unitaire_fcfa DESC
    LIMIT 10
    """).show(truncate=False)

```

```

+-----+-----+
|nom_produit|prix_fcfa|
+-----+-----+
|Hisense Informatique 0593|844000|
|Royal Informatique 0439|823500|
|LG Informatique 0469|813500|
|Kirène Informatique 0577|707000|
|Adidas Informatique 0383|698000|
|HP Électroménager 0377|587500|
|Sunu Tech Informatique 0340|587000|
|Sunu Tech Informatique 0180|586000|
|Infinix Informatique 0445|584000|
|Samsung Électroménager 0063|571000|
+-----+-----+

```

B4 — Commandes livrées, canal mobile, décembre 2025

Combien de commandes `livree` du canal `mobile_app` en décembre 2025 ?

```
In [21]: spark.sql("""
SELECT COUNT(*) AS nb_commandes
FROM commandes
WHERE statut = 'livrée'
  AND canal = 'mobile_app'
  AND date_commande >= '2025-12-01'
  AND date_commande < '2026-01-01'
""").show()
```

```
+-----+
|nb_commandes|
+-----+
|          1745|
+-----+
```

B5 — Emails manquants

Combien de clients ont un email `NULL` ou égal à `'N/A'` ? (Rappel séance 3 : le manquant a deux visages — et `= NULL` ne fonctionne pas.)

```
In [22]: # === À COMPLÉTER ===
spark.sql("""
SELECT COUNT(*) AS nb_emails_manquants
FROM clients
WHERE email IS NULL
  OR email = 'N/A'
""").show()
```

```
+-----+
|nb_emails_manquants|
+-----+
|                150|
+-----+
```

Partie C — L'enquête FCFA (30 min)

C1 — Le symptôme : la somme naïve

Calculez le CA total directement sur la colonne brute, et **notez le résultat**.

```
In [23]: # === À COMPLÉTER ===
ca_naif = spark.sql("""
SELECT SUM(montant_total_fcfa) AS ca FROM commandes
""").first()["ca"]
relevés["C1_ca_naif"] = ca_naif
print(f"CA naif : {ca_naif:,.0f}")
```

CA naif : 11,519,493,000

C1 — CA naïf : 11 519 493 000 FCFA

C2 — Diagnostiquer

Comptez les valeurs de `montant_total_fcfa` qui ne sont **pas** de purs nombres, puis affichez 10 valeurs fautives distinctes.

```
In [24]: # === À COMPLÉTER ===
nb_pollues = spark.sql("""
    SELECT COUNT(*) AS nb
    FROM commandes
    WHERE montant_total_fcfa NOT rlike '^[0-9]+$'
""").first()["nb"]

relevés["C2_nb_valeurs_polluees"] = nb_pollues
print("valeurs polluees :", nb_pollues)

spark.sql("""
    SELECT DISTINCT montant_total_fcfa
    FROM commandes
    WHERE montant_total_fcfa NOT rlike '^[0-9]+$'
    LIMIT 10
""").show(truncate=False)
```

valeurs polluees : 500

```
+-----+
|montant_total_fcfa|
+-----+
|845500 FCFA      |
|554000 FCFA      |
|87700 FCFA       |
|21000 FCFA       |
|245500 FCFA      |
|154000 FCFA      |
|187000 FCFA      |
|57000 FCFA       |
|56700 FCFA       |
|1123900 FCFA     |
+-----+
```

Question C — Expliquez en deux phrases pourquoi la requête C1 rend un résultat **faux sans lever d'erreur**.

Votre réponse : La requête C1 donne un résultat faux car la colonne `montant_total_fcfa` contient 500 valeurs polluées avec le format "montant FCFA" au lieu d'un nombre pur. Spark ne déclenche pas d'erreur car il ignore ces valeurs non numériques lors de l'agrégation, ce qui entraîne un calcul du chiffre d'affaires incomplet.

C3 — Nettoyer : la vue `commandes_clean`

Complétez la regex : supprimer **tout ce qui n'est pas un chiffre**, puis caster en `BIGINT`. On conserve la colonne brute sous `montant_raw`.

```
In [25]: # === À COMPLÉTER ===
spark.sql("""
```

```
CREATE OR REPLACE TEMP VIEW commandes_clean AS
SELECT order_id, customer_id, date_commande, statut, canal,
       frais_livraison_fcfa,
       montant_total_fcfa AS montant_raw,
       CAST(regex_replace(trim(montant_total_fcfa),
                           '^[^0-9]', '' ) AS BIGINT) AS montant_fcfa
FROM commandes
""")

spark.sql("SELECT montant_raw, montant_fcfa FROM commandes_clean LIMIT 5").show()
```

```
+-----+-----+
|montant_raw|montant_fcfa|
+-----+-----+
|      190500|      190500|
|       32200|       32200|
|       40500|       40500|
|        4000|        4000|
|       35500|       35500|
+-----+-----+
```

C4 — Valider : mesurer, pas affirmer

Vérifiez qu'aucun `NULL` n'a été produit, contrôlez `MIN` / `MAX`, et recalculez le CA.

```
In [26]: # === À COMPLÉTER ===
validation = spark.sql("""
    SELECT COUNT(*)                AS nb_lignes,
           COUNT(montant_fcfa)      AS nb_castes,
           COUNT(*) - COUNT(montant_fcfa) AS nb_null,
           MIN(montant_fcfa)        AS mini,
           MAX(montant_fcfa)        AS maxi,
           SUM(montant_fcfa)        AS ca_total
    FROM commandes_clean
""")

validation.show()

releves["C4_ca_nettoye"] = validation.first()["ca_total"]
```

```
+-----+-----+-----+-----+-----+-----+
|nb_lignes|nb_castes|nb_null|mini|  maxi|  ca_total|
+-----+-----+-----+-----+-----+-----+
|    50000|    50000|      0| 500|4182000|11645231000|
+-----+-----+-----+-----+-----+-----+
```

C5 — La preuve chiffrée

Calculez l'écart entre le CA naïf (C1) et le CA nettoyé (C4), en FCFA et en pourcentage.

```
In [27]: # === À COMPLÉTER ===
ecart = releves["C4_ca_nettoye"] - releves["C1_ca_naif"]
releves["C5_ecart_fcfa"] = ecart
releves["C5_ecart_pct"]  = (ecart / releves["C1_ca_naif"]) * 100

print(f"Ecart : {ecart:,.0f} FCFA soit {releves['C5_ecart_pct']:.2f} %")
```


Ecart : 125,738,000 FCFA soit 1.09 %

Partie D — Les indicateurs de la direction (40 min)

Toutes les requêtes portent sur `commandes_clean` (et `paiements` pour D5). Après chaque résultat, ajoutez **une phrase d'interprétation métier** dans la cellule markdown qui suit.

D1 — CA et commandes par statut

Quelle part du CA est réellement `livree` ?

```
In [28]: # === À COMPLÉTER ===
spark.sql("""
    SELECT statut,
           COUNT(*)           AS nb_commandes,
           SUM(montant_fcfa) AS ca_fcfa
    FROM commandes_clean
    GROUP BY statut
    ORDER BY ca_fcfa DESC
""").show()

releases["D1_part_ca_livree_pct"] = (
    spark.sql("""
        SELECT SUM(CASE WHEN statut = 'livree' THEN montant_fcfa ELSE 0 END) * 1
              / SUM(montant_fcfa) AS part_ca_livree_pct
        FROM commandes_clean
        """).first()["part_ca_livree_pct"]
    )
```

statut	nb_commandes	ca_fcfa
livrée	38890	9090757800
annulée	4568	1066017300
en_cours	4058	911567000
retournée	2484	576888900

Votre réponse :

La part du chiffre d'affaires réellement livrée est de 78,07 %. Cela signifie qu'environ 78 % du CA provient des commandes effectivement livrées, tandis que le reste correspond aux commandes annulées, en cours ou retournées.

D2 — Le CA mensuel des commandes livrées

`date_trunc('month', ...)`, tri chronologique. Repérez la tendance et le mois record.

```
In [28]: # === À COMPLÉTER ===
ca_mensuel = spark.sql("""
    SELECT date_trunc('month', date_commande) AS mois,
           COUNT(*)                           AS nb_commandes,
           SUM(montant_fcfa)                   AS ca_fcfa
    FROM commandes_clean
    GROUP BY mois
    ORDER BY ca_fcfa DESC
""").show()
```

```

FROM commandes_clean
WHERE montant_fcfa IS NOT NULL
GROUP BY date_trunc('month', date_commande)
ORDER BY mois
""")

ca_mensuel.show(24, truncate=False)

relevés["D2_mois_record"] = (
    ca_mensuel
    .orderBy("ca_fcfa", ascending=False)
    .first()["mois"]
)

```

mois	nb_commandes	ca_fcfa
2024-07-01 00:00:00	1593	383318100
2024-08-01 00:00:00	1579	364931600
2024-09-01 00:00:00	1696	387541700
2024-10-01 00:00:00	1710	407378900
2024-11-01 00:00:00	1677	386586300
2024-12-01 00:00:00	2715	614704900
2025-01-01 00:00:00	1836	432125800
2025-02-01 00:00:00	1678	386808600
2025-03-01 00:00:00	1885	425659500
2025-04-01 00:00:00	1810	421293800
2025-05-01 00:00:00	1947	468795800
2025-06-01 00:00:00	1954	461670100
2025-07-01 00:00:00	2119	467115500
2025-08-01 00:00:00	2044	496188400
2025-09-01 00:00:00	2052	486799500
2025-10-01 00:00:00	2171	518114200
2025-11-01 00:00:00	2119	468888600
2025-12-01 00:00:00	3431	800856100
2026-01-01 00:00:00	2258	543725700
2026-02-01 00:00:00	2129	493652100
2026-03-01 00:00:00	2307	534543900
2026-04-01 00:00:00	2384	558566800
2026-05-01 00:00:00	2524	584857600
2026-06-01 00:00:00	2382	551107500

Votre réponse :

Le mois record est décembre 2025 avec un chiffre d'affaires de 800 856 100 FCFA pour 3 431 commandes. Cette performance indique une forte activité commerciale durant cette période.

D3 — Panier moyen par canal

`ROUND(AVG(montant_fcfa), 0)` — mobile ou web, qui dépense le plus par commande ?

```

In [29]: # === À COMPLÉTER ===
panier = spark.sql("""
SELECT canal,

```

```

        COUNT(*)                AS nb_commandes,
        SUM(montant_fcfa)        AS ca_fcfa,
        ROUND(AVG(montant_fcfa), 0) AS panier_moyen_fcfa
    FROM commandes_clean
    GROUP BY canal
    ORDER BY ca_fcfa DESC
    """
)
panier.show()

releves["D3_panier_moyen_mobile"] = (
    panier.filter("canal = 'mobile_app'")
    .first()["panier_moyen_fcfa"]
)

```

canal	nb_commandes	ca_fcfa	panier_moyen_fcfa
mobile_app	32519	7591773700	233457.0
web	17481	4053457300	231878.0

Votre réponse :

Le canal mobile_app présente un panier moyen de 233 457 FCFA, légèrement supérieur à celui du canal web (231 878 FCFA). Cela montre que les clients utilisant l'application mobile dépensent en moyenne un peu plus par commande.

D4 — Top clients, avec HAVING

Top 10 des clients par CA (`GROUP BY customer_id`), en ne gardant que les clients dépassant **1 000 000 FCFA** de CA cumulé. Un client sort-il du lot ?

```

In [30]: # === À COMPLÉTER ===
spark.sql("""
    SELECT customer_id,
           COUNT(*)      AS nb_commandes,
           SUM(montant_fcfa) AS ca_fcfa
    FROM commandes_clean
    GROUP BY customer_id
    HAVING SUM(montant_fcfa) > 1000000
    ORDER BY ca_fcfa DESC
    LIMIT 10
    """).show()

```

customer_id	nb_commandes	ca_fcfa
C000001	3143	732953000
C000002	421	95219500
C000003	323	70031000
C000005	252	69692400
C000004	284	66843700
C000006	235	53352600
C000009	162	46914400
C000008	182	44441800
C000007	180	39903700
C000013	135	39245500

```
In [32]: spark.sql("""
SELECT c.customer_id,
       c.nom,
       c.prenom,
       COUNT(*)          AS nb_commandes,
       SUM(cc.montant_fcfa) AS ca_fcfa
FROM commandes_clean cc
JOIN clients c
  ON cc.customer_id = c.customer_id
GROUP BY c.customer_id, c.nom, c.prenom
HAVING SUM(cc.montant_fcfa) > 1000000
ORDER BY ca_fcfa DESC
LIMIT 10
""").show()
```

customer_id	nom	prenom	nb_commandes	ca_fcfa
C000001	Ba	Aïssatou	3143	732953000
C000002	Diop	Malick	421	95219500
C000003	Ndour	Souleymane	323	70031000
C000005	Dieng	Coumba	252	69692400
C000004	Mbodj	Coumba	284	66843700
C000006	Tall	Assane	235	53352600
C000009	Samb	Moussa	162	46914400
C000008	Badji	Bineta	182	44441800
C000007	Kane	Aïssatou	180	39903700
C000013	Camara	Yacine	135	39245500

Votre réponse :

Le client C000001 sort largement du lot avec un chiffre d'affaires cumulé de 732 953 000 FCFA pour 3 143 commandes. Il représente un écart très important par rapport aux autres clients, ce qui indique une forte contribution de ce client au chiffre d'affaires global. J'ai exécuté un autre code pour savoir exactement le nom et le prénom du concerné. Il s'agit de Aïssatou Ba.

D5 — Paiements par méthode

Nombre et pourcentage par méthode. Quelle part totale pour le **mobile money** (Orange Money + Wave + Free Money) ?

```
In [16]: # === À COMPLÉTER ===
spark.sql("""
    SELECT methode,
           COUNT(*) AS nb,
           ROUND(100 * COUNT(*) / SUM(COUNT(*) OVER (), 1) AS part_pct
    FROM paiements
    GROUP BY methode
    ORDER BY nb DESC
""").show()

releases["D5_part_mobile_money_pct"] = (
    spark.sql("""
        SELECT ROUND(
            100 * COUNT(*) / (SELECT COUNT(*) FROM paiements), 1
        ) AS part_pct
        FROM paiements
        WHERE methode IN ('Wave', 'Orange Money', 'Free Money')
    """).first()["part_pct"]
)
```

methode	nb	part_pct
Orange Money	15646	35.1
Wave	13400	30.1
Paielement à la liv...	9783	22.0
Carte bancaire	3506	7.9
Free Money	2228	5.0

Votre réponse :

Le mobile money (Orange Money, Wave et Free Money) représente 70,2 % des paiements. Cela montre que les clients privilégient largement les solutions de paiement mobile, qui constituent le principal moyen de paiement.

Partie E — SQL ou API ? `explain()` tranche (20 min)

E1 — D3 en API DataFrame

Réécrivez le panier moyen par canal avec `groupBy().agg()`. Les chiffres doivent être **identiques**.

Suggestion : Il faut 'abord créer une vue `commandes_clean` qui contient les colonnes suivantes : `order_id`, `customer_id`, `date_commande`, `statut`, `canal`, `frais_livraison_fcfa`, `montant_total_fcfa` AS `montant_raw`, `CAST(regexp_replace(trim(montant_total_fcfa), '^[0-9]', '') AS BIGINT)` AS `montant_fcfa`

```
In [4]: spark.sql("""
CREATE OR REPLACE TEMP VIEW commandes_clean AS
SELECT
    order_id,
    customer_id,
    date_commande,
    statut,
    canal,
    frais_livraison_fcfa,
    montant_total_fcfa AS montant_raw,
    CAST(regexp_replace(trim(montant_total_fcfa), '^[^0-9]', '') AS BIGINT) AS mo
FROM commandes
""")
```

Out[4]: DataFrame[]

```
In [5]: from pyspark.sql import functions as F

# === À COMPLÉTER ===
commandes_clean_df = spark.table("commandes_clean")

panier_api = (commandes_clean_df
    .groupBy("canal")
    .agg(
        F.count("*").alias("nb_commandes"),
        F.sum("montant_fcfa").alias("ca_fcfa"),
        F.round(F.avg("montant_fcfa"), 0).alias("panier_moyen_fcfa")
    )
    .orderBy(F.col("ca_fcfa").desc())
)

panier_api.show()
```

```
+-----+-----+-----+-----+
| canal|nb_commandes| ca_fcfa|panier_moyen_fcfa|
+-----+-----+-----+-----+
|mobile_app|      32519|7591773700|      233457.0|
|      web|      17481|4053457300|      231878.0|
+-----+-----+-----+-----+
```

In []:

E2 — B4 en API

La même requête « livrées / mobile / décembre 2025 », version `filter`.

```
In [7]: # === À COMPLÉTER ===
nb_api = (spark.table("commandes")
    .filter(
        (F.col("statut") == "livrée") &
        (F.col("canal") == "mobile_app") &
        (F.col("date_commande") >= "2025-12-01") &
        (F.col("date_commande") < "2026-01-01")
    )
    .count())
print(nb_api)
```

1745

E3 — Comparer les plans

Affichez le plan physique de la version SQL de D3 et de `panier_api`.

```
In [30]: panier_sql = spark.sql("""
        SELECT canal, COUNT(*) AS nb_commandes,
               SUM(montant_fcfa) AS ca_fcfa,
               ROUND(AVG(montant_fcfa), 0) AS panier_moyen_fcfa
        FROM commandes_clean
        GROUP BY canal
        ORDER BY ca_fcfa DESC
        """)
panier_sql.explain()
panier_api.explain()

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [ca_fcfa#291L DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(ca_fcfa#291L DESC NULLS LAST, 200), ENSURE_REQUIREMENTS, [plan_id=448]
      +- HashAggregate(keys=[canal#66], functions=[count(1), sum(montant_fcfa#302L), avg(montant_fcfa#302L)])
         +- Exchange hashpartitioning(canal#66, 200), ENSURE_REQUIREMENTS, [plan_id=445]
            +- HashAggregate(keys=[canal#66], functions=[partial_count(1), partial_sum(montant_fcfa#302L), partial_avg(montant_fcfa#302L)])
               +- Project [canal#66, cast(regex_replace(trim(montant_total_fcfa#68, None), [^0-9], , 1) as bigint) AS montant_fcfa#302L]
                  +- FileScan csv [canal#66,montant_total_fcfa#68] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/c:/Users/24kha/Desktop/MSIA_ISI/TP BIG DATA/data/orders.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<canal:string,montant_total_fcfa:string>

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [ca_fcfa#248L DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(ca_fcfa#248L DESC NULLS LAST, 200), ENSURE_REQUIREMENTS, [plan_id=470]
      +- HashAggregate(keys=[canal#66], functions=[count(1), sum(montant_fcfa#246L), avg(montant_fcfa#246L)])
         +- Exchange hashpartitioning(canal#66, 200), ENSURE_REQUIREMENTS, [plan_id=467]
            +- HashAggregate(keys=[canal#66], functions=[partial_count(1), partial_sum(montant_fcfa#246L), partial_avg(montant_fcfa#246L)])
               +- Project [canal#66, cast(regex_replace(trim(montant_total_fcfa#68, None), [^0-9], , 1) as bigint) AS montant_fcfa#246L]
                  +- FileScan csv [canal#66,montant_total_fcfa#68] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/c:/Users/24kha/Desktop/MSIA_ISI/TP BIG DATA/data/orders.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<canal:string,montant_total_fcfa:string>
```

Question E — Les plans physiques sont-ils identiques ? Où voit-on le filtre poussé vers la lecture (`PushedFilters` / `Filter` près du `FileScan`) ? Que concluez-vous sur le choix SQL vs API ? (3 phrases)

Votre réponse :

Les deux plans physiques sont quasiment identiques : ils utilisent le même enchaînement d'opérations Spark (FileScan CSV → Project pour nettoyer et convertir montant_total_fcfa → HashAggregate → Sort), avec seulement des identifiants internes différents (plan_id, numéros de colonnes).

On remarque qu'il n'y a aucun filtre poussé vers la lecture : dans le FileScan, on voit PushedFilters: [] et il n'y a pas d'opération Filter avant l'agrégation. Cela signifie que Spark lit toutes les lignes du fichier CSV avant d'effectuer les transformations.

On conclut que le choix entre SQL et API DataFrame n'a pas d'impact sur le plan d'exécution dans ce cas : Spark Catalyst optimise les deux approches pour produire un plan physique équivalent. Le choix dépend donc surtout de la lisibilité, des habitudes du développeur et du besoin d'utiliser des transformations programmatiques.

Partie F — Discussion guidée (10 min, en groupes)

1. Le CA naïf de C1 était faux **en silence**. Dans une vraie entreprise, qui s'en serait aperçu, quand, et à quel coût ? Proposez **deux garde-fous techniques**.
2. `commandes_clean` est une vue temporaire : que se passe-t-il demain matin au redémarrage du notebook ? Est-ce acceptable en production ? (indice : séances 6-7)
3. La direction veut le CA par **ville du client** : quelle information manque à `commandes_clean` seule, et comment l'obtiendrez-vous en séance 5 ?

Votre réponse :

1. CA naïf faux en silence

Dans une vraie entreprise, l'erreur pourrait être détectée par le service financier lors d'un contrôle ou d'un rapprochement avec les données de paiement, mais parfois seulement après la publication des rapports, ce qui pourrait entraîner de mauvaises décisions et des pertes financières. Deux garde-fous techniques sont : valider automatiquement le type et le format des montants avant les calculs, et mettre en place des contrôles de qualité des données avec des alertes lorsque des valeurs anormales sont détectées.

2. Vue temporaire commandes_clean

Au redémarrage du notebook ou de la session Spark, la vue temporaire commandes_clean disparaît et doit être recréée. Ce fonctionnement n'est pas adapté à la production car les traitements doivent être reproductibles et persistants ; il faudrait plutôt utiliser une table persistante ou intégrer le nettoyage dans un pipeline de données automatisé.

3. CA par ville du client

commandes_clean contient le customer_id, mais pas la ville du client. Il faut donc faire une jointure avec la table clients grâce à customer_id afin de récupérer la ville, puis

regrouper les commandes par ville pour calculer le CA.

Quiz éclair (10 min)

1. Que retourne `spark.sql(...)` : une liste, un DataFrame ou un fichier ?
2. `createOrReplaceTempView` copie-t-elle les données ? Quelle est la portée de la vue ?
3. Pourquoi `WHERE email = NULL` ne renvoie-t-il jamais rien ?
4. `SUM` sur une colonne `string` polluée : erreur ou résultat faux ? Pourquoi est-ce dangereux ?
5. `WHERE` et `HAVING` : lequel filtre les groupes, lequel filtre les lignes ?

Notez votre score dans la cellule suivante.

Votre réponse :

1. `spark.sql(...)` retourne un DataFrame Spark.
2. `createOrReplaceTempView` ne copie pas les données : elle crée une vue temporaire basée sur le DataFrame. Sa portée est limitée à la session Spark.
3. `NULL` représente une valeur inconnue en SQL. Il faut utiliser `IS NULL` au lieu de `= NULL`.
4. Avec une colonne `string` polluée, le calcul peut produire un résultat faux sans erreur, ce qui est dangereux car on peut prendre une mauvaise valeur pour un résultat correct.
5. `WHERE` filtre les lignes avant le regroupement, tandis que `HAVING` filtre les groupes après `GROUP BY`.

Pour finir : relevés et livrable

```
In [21]: print("=" * 60)
print("TABLEAU DE RELEVES - TP4")
print("=" * 60)
for k, v in releves.items():
    print(f"{k:32s} : {v}")

manquants = [k for k, v in releves.items() if v is None]
print("\nRelevés manquants :", manquants if manquants else "aucun - bravo !")
```

```
=====
TABLEAU DE RELEVES – TP4
=====
A_nb_lignes_clients           : 5025
A_nb_lignes_commandes         : 50000
A_type_montant_total_fcfa      : string
C2_nb_valeurs_polluees        : None
C1_ca_naif                    : None
C4_ca_nettoye                 : None
C5_ecart_fcfa                 : None
C5_ecart_pct                  : None
D1_part_ca_livree_pct         : None
D2_mois_record                : None
D3_panier_moyen_mobile        : None
D5_part_mobile_money_pct      : 70.2
```

Relevés manquants : ['C2_nb_valeurs_polluees', 'C1_ca_naif', 'C4_ca_nettoye', 'C5_ecart_fcfa', 'C5_ecart_pct', 'D1_part_ca_livree_pct', 'D2_mois_record', 'D3_panier_moyen_mobile']

Pousser le livrable

Depuis la racine de votre dépôt :

```
git status           # verifier que data/ n'apparait
PAS
git add notebooks/TP4_spark_sql.ipynb
git commit -m "TP4 : requetes SQL et nettoyage FCFA"
git push
```

Checklist finale

- ☐ Notebook exécuté de bout en bout (Restart & Run All), sorties visibles ;
- ☐ `nb_null = 0` dans la validation C4 ;
- ☐ Écart CA naïf / nettoyé relevé en FCFA **et** en % ;
- ☐ Une phrase d'interprétation sous chaque indicateur de la partie D ;
- ☐ Aucun relevé manquant dans la cellule ci-dessus ;
- ☐ Données non committées.

Séance 5 : les jointures — lecture préalable : Damji et al., Learning Spark 2e éd., chapitre 5.